Pippin: Algorithm designs

Last updated 23 October 2026. Owners: Shea Palin and Leeam.

This is Brief Task 1, Activity B. It sets out five of the more complex problems on T Level Connect as pseudocode. Designs 1, 2 and 5 match the code as it is now (main.py, chat_api.py and topNavBar.js). Designs 3 and 4 haven't been built yet, so they're the plan for building them.

Pseudocode conventions: <- means assignment, and blocks close with END IF, END FOR and so on. RETURN ends the algorithm. RESPOND sends an HTTP status and body back to the browser. Text in quotes is the message the user sees.

1. Register and login validation

Problem: create accounts and sign people in safely. Bad input is rejected with a clear message, passwords are never stored as plain text, a failed sign-in doesn't reveal which emails are registered, and users are only sent back to pages on this site. Inputs: the form fields and CSRF token. Outputs: a signed-in session and a redirect, or the form again with one error.

```
CONSTANTS MAX_NAME <- 100, MAX_EMAIL <- 254, MIN_PW <- 8, MAX_PW <- 128

FUNCTION Register(form)
    IF CSRF token missing or invalid THEN RESPOND 400 "Form expired, refresh and try again"
    IF more than 10 attempts from this IP in the last minute THEN RESPOND 429
    name <- TRIM(form.name)
    email <- LOWERCASE(TRIM(form.email))
    password <- form.password            // not trimmed, spaces may be deliberate
    IF name = "" OR email = "" OR password = "" THEN RETURN "Please fill in every field."
    IF LENGTH(name) > MAX_NAME THEN RETURN "Name must be 100 characters or fewer."
    IF LENGTH(email) > MAX_EMAIL OR email does not match "text@text.text" (no spaces) THEN
        RETURN "Please enter a valid email address."
    END IF
    IF LENGTH(password) < MIN_PW THEN RETURN "Password must be at least 8 characters."
    IF LENGTH(password) > MAX_PW THEN RETURN "Password must be 128 characters or fewer."
    IF password <> form.confirm_password THEN RETURN "Passwords do not match."
    hash <- PBKDF2_SHA256(password)      // salted, never store the real password
    TRY
        userId <- INSERT INTO users (name, email, hash)
    CATCH unique-email error
        RETURN "An account with that email already exists. Please sign in instead."
    END TRY
    CLEAR session                        // stops an old session being reused
    session.user_id <- userId
    REDIRECT to /account
END FUNCTION

FUNCTION Login(form, nextPage)
    (same CSRF and rate-limit checks as Register)
    email <- LOWERCASE(TRIM(form.email))
    user <- SELECT * FROM users WHERE email = email
    IF user = NULL OR NOT CheckHash(user.hash, form.password) THEN
        RETURN "Incorrect email or password."     // same message for both cases
    END IF
    CLEAR session
    session.user_id <- user.id
    IF IsSafeRedirect(nextPage) THEN REDIRECT to nextPage ELSE REDIRECT to /account
END FUNCTION

FUNCTION IsSafeRedirect(target)
    IF target = "" OR target does not start with "/" THEN RETURN FALSE
    target <- REPLACE "\" WITH "/" IN target       // browsers treat them the same
    parts <- PARSE_URL(target)
    RETURN parts.scheme = "" AND parts.domain = ""  // a path on this site only
END FUNCTION
```

Test data: "Bob@X.com " gets stored as "bob@x.com". A 7-character password is rejected. Registering the same email twice gives the "already exists" message. A wrong password and an unknown email give the same message. A next value of "//evil.com" or "/\evil.com" goes to /account instead.

2. Chatbot request flow

Problem: pass a visitor's conversation to the AI model (OpenRouter) and stream the reply back, keeping the API key secret, stopping overload and misuse, stopping visitors overriding the assistant's instructions, and never showing the model's hidden reasoning. Inputs: JSON {messages: [{role, content}, ...]}. Outputs: a stream of {delta}, then {done} or {error}.

```
CONSTANTS LIMIT <- 10 per 60 seconds, MAX_BODY <- 100 KB, MAX_MSGS <- 20,
          MAX_CHARS <- 4000, MAX_STREAM_SECONDS <- 120

FUNCTION ChatStream(request)
    // 1. Rate limit (sliding window per IP address, shared with /api/chat)
    hits <- recentRequests[request.ip]
    REMOVE from hits every time older than 60 seconds
    IF COUNT(hits) >= LIMIT THEN RESPOND 429 "You're sending messages too quickly" + Retry-After
    ADD now TO hits

    // 2. Size and shape
    IF request.size > MAX_BODY THEN RESPOND 413 "Your message is too long."
    messages <- PARSE_JSON(request.body)
    IF messages is not a list of 1 to MAX_MSGS items THEN RESPOND 400
    clean <- empty list
    FOR EACH m IN messages
        IF m.role NOT IN ("user", "assistant") THEN RESPOND 400   // no "system" messages
        text <- TRIM(m.content)
        IF text = "" OR LENGTH(text) > MAX_CHARS THEN RESPOND 400
        APPEND {role: m.role, content: text} TO clean               // extra fields dropped
    END FOR
    IF LAST(clean).role <> "user" THEN RESPOND 400

    // 3. Call the model
    IF API key not set THEN RESPOND 503 "The chat assistant is not available"
    body <- {model, [SYSTEM_PROMPT] + clean, max_tokens 4096,
             reasoning {effort low, exclude}, stream TRUE}
    TRY
        upstream <- POST body TO OpenRouter (connect timeout 5 s, read timeout 90 s)
    CATCH timeout: RESPOND 504    CATCH connection error: RESPOND 502
    END TRY
    IF upstream.status = 429 THEN RESPOND 503 "The assistant is busy"
    IF upstream.status is not 2xx THEN RESPOND 502   // details go to the log only

    // 4. Relay the stream
    START event stream to browser
    started <- FALSE, finished <- FALSE, deadline <- now + MAX_STREAM_SECONDS
    FOR EACH line IN upstream (decoded as UTF-8)
        IF now > deadline THEN SEND {error: "took too long"}; STOP
        IF line is blank or a comment THEN CONTINUE
        IF line = "data: [DONE]" THEN finished <- TRUE; EXIT FOR
        chunk <- PARSE_JSON(line) (if this fails, CONTINUE)
        IF chunk has error THEN SEND {error: "temporarily unavailable"}; STOP
        text <- chunk.choices[0].delta.content    // reasoning is never read
        IF text <> "" THEN
            IF NOT started THEN text <- TRIM_START(text)
            IF text <> "" THEN started <- TRUE; SEND {delta: text}
        END IF
        IF chunk has finish_reason THEN finished <- TRUE
    END FOR
    ALWAYS close upstream connection
    IF NOT started THEN SEND {error: "couldn't come up with a reply"}
    ELSE IF NOT finished THEN SEND {error: "reply was interrupted"}
    ELSE SEND {done: TRUE}
END FUNCTION
```

The browser tries the stream first and falls back to /api/chat if streaming fails. Test data: an 11th request within a minute gets a 429. A message with role "system" gets a 400. A conversation ending on an assistant message gets a 400. If the connection drops mid-reply, the visitor sees "interrupted", not half an answer.

3. Expression of interest submission (planned)

Problem: let a signed-in user register interest in a placement route, and change it later, without creating duplicates. It should only store what's needed, and only when the user has agreed to be contacted. Inputs: JSON {route, role, school (optional), contact_consent} plus the X-CSRFToken header. Outputs: 201 if created, 200 if updated, or an error.

```
CONSTANTS ROUTES <- (digital, business, media, finance, engineering)
          ROLES  <- (student, parent_carer, teacher_adviser, other), MAX_SCHOOL <- 100

FUNCTION SubmitInterest(request)
    IF session has no user THEN RESPOND 401 {error: "Please sign in"}  // JSON, not a redirect
    IF CSRF header missing or invalid THEN RESPOND 400
    IF more than 10 attempts from this user in the last minute THEN RESPOND 429
    data <- PARSE_JSON(request.body)
    IF data is not an object THEN RESPOND 400 {error: "Expected a JSON object"}
    route <- LOWERCASE(TRIM(data.route))
    role <- LOWERCASE(TRIM(data.role))
    school <- TRIM(data.school OR "")
    IF route NOT IN ROUTES THEN RESPOND 400 {error: "Unknown route"}
    IF role NOT IN ROLES THEN RESPOND 400 {error: "Unknown role"}
    IF LENGTH(school) > MAX_SCHOOL THEN RESPOND 400 {error: "School name too long"}
    IF data.contact_consent IS NOT TRUE THEN RESPOND 400 {error: "Consent is required"}

    existing <- SELECT FROM interests WHERE user_id = user.id AND route = route
    IF existing = NULL THEN
        INSERT INTO interests (user.id, route, role, school, TRUE, now, now)
        RESPOND 201 {route, role, school}
    ELSE
        UPDATE interests SET role, school, updated_at = now WHERE id = existing.id
        RESPOND 200 {route, role, school}
    END IF
END FUNCTION
```

The table has UNIQUE(user_id, route), so if two requests arrive at the same moment the second one updates the row instead of making a duplicate. DELETE /api/interest/<route> uses the same sign-in and CSRF checks, removes the row and responds 204. Test data: a route of "Digital " is accepted as "digital". A route of "law" gets a 400. Sending the same route twice gives 201 then 200, and there's still only one row. A request with no session gets a 401.

4. Gated download access check (planned)

Problem: serve resources like CV templates and placement guides. Public files are open to everyone, but members-only files need the user to be signed in. Nobody should be able to use the download link to read other files on the server. Inputs: a resource id in the URL, plus the session. Outputs: the file, or 401, 404 or 429.

```
FUNCTION DownloadResource(id, request)
    IF more than 30 downloads from this IP in the last minute THEN RESPOND 429
    IF id is not a whole number THEN RESPOND 404
    resource <- SELECT FROM resources WHERE id = id
    IF resource = NULL THEN RESPOND 404 {error: "Resource not found"}
    IF resource.members_only AND session has no user THEN
        RESPOND 401 {error: "Please sign in to download this"}
    END IF
    // The file name comes from the database, never from the URL
    path <- JOIN(RESOURCES_FOLDER, resource.filename)
    IF path is outside RESOURCES_FOLDER OR file does not exist THEN
        LOG "missing resource file", id
        RESPOND 404 {error: "Resource not found"}
    END IF
    RESPOND 200 with file (Content-Type from resource.file_type,
                           Content-Disposition: attachment,
                           Cache-Control: private if members_only)
END FUNCTION
```

The sign-in check comes after the 404 check, so a 401 does give away that the id exists. That's fine, because the resource list already shows every title. Members-only files are marked Cache-Control: private so a shared cache can't pass them to someone who isn't signed in. Test data: a public file with no session gets a 200. A members-only file with no session gets a 401, and with a session gets a 200. An id of 9999 gets a 404. An id of "../main.py" gets a 404.

5. Site search

Problem: the nav bar's search box should take any words the user types and go straight to the most relevant page or section. It has to cope with capitals, plurals, partial words and filler words, and give a helpful message if nothing matches. Inputs: the query text. Outputs: a jump to the best entry in SEARCH_INDEX (title, path, keywords), or a message.

```
FUNCTION Normalise(text)
    RETURN LOWERCASE(text) with punctuation replaced by spaces and repeated spaces removed
END FUNCTION

FUNCTION Words(text)
    RETURN words of Normalise(text) that are not stop words ("the", "a", "how"...)
END FUNCTION

FUNCTION ScoreEntry(entry, query)
    q <- Normalise(query)
    IF q = "" THEN RETURN 0
    IF Normalise(entry.title) = q THEN RETURN 1000            // exact title wins
    short <- entry.title without a "FAQ: " style label
    titleWords <- Words(short), keywordWords <- Words(entry.keywords)
    score <- 0
    IF q appears inside Normalise(short) THEN score <- score + 50
    IF q has several words AND q appears as a phrase in keywords THEN score <- score + 40
    FOR EACH w IN Words(query)
        IF w IN titleWords THEN score <- score + 10
        ELSE IF w IN keywordWords THEN score <- score + 6
        ELSE IF LENGTH(w) >= 3 AND some k IN titleWords + keywordWords
                (k starts with w OR (LENGTH(k) >= 4 AND w starts with k)) THEN
            score <- score + 3                                 // "eng", "placements"
        END IF
    END FOR
    IF score > 0 AND entry.path has no "#section" THEN score <- score + 1  // page beats its section
    RETURN score
END FUNCTION

FUNCTION Search(query)
    query <- TRIM(query)
    IF query = "" THEN SHOW 'Type a word to search, for example "digital"'; RETURN
    best <- NULL, bestScore <- 0
    FOR EACH entry IN SEARCH_INDEX
        s <- ScoreEntry(entry, query)
        IF s > bestScore THEN best <- entry; bestScore <- s      // first entry wins a tie
    END FOR
    IF best = NULL THEN SHOW 'No results for "query". Try "T Levels" or "contact"'; RETURN
    CLEAR search box
    IF best opens the accessibility menu THEN OPEN menu
    ELSE IF best is the section already on screen THEN SCROLL to it
    ELSE GO TO best.path
END FUNCTION
```

Each search checks every entry once, so it takes time in proportion to the size of the index. With 23 entries that's instant, and it runs in the browser without contacting the server. Test data: "FAQ" goes to the FAQ page itself, not one of its sections. "eng" goes to the Engineering route. "coding" finds Digital through its keywords. "xyzzy" shows the no-results message. An empty box shows the hint.